Объектно-ориентированное программирование в python

Этот сайт посвящен особенностям ООП в python. Для открытия у себя на устройстве можете отсканировать qr:

Класс

Python сильно более объектно ориентирован, чем многие языки. Почти все что вы встретите в python будет объектом.
Функции, числа, классы, модули - это все объекты.

По сути тип данных в python - это класс объекта. Так что для получения класса объекта есть `type`

```python
type(1)
```

```
<class 'int'>
```

```python
type([1, "2", (3,)])
```

```
<class 'list'>
```

```python
class A: ...
type(A())
```

```
<class '__main__.A'>
```

Но в python как говорилось раньше практически все - это объект К примеру:

```python
type(A)
```

```
<class 'type'>
```

```python
type(type)
```

```
<class 'type'>
```

```python
def func(): ...
type(func)
```

```
<class 'function'>
```

```python
type(type(func))
```

```
<class 'type'>
```

Как мы знаем у классов есть атрибуты и методы.

На уровне реализации языка внутри каждый объект - это struct в Си, которая содержит:
- Счетчик ссылок: `ob_refcnt`
- Указатель на тип: `ob_type`

И выглядит примерно так:

```c
typedef struct _object {
    _PyObject_HEAD_EXTRA
    Py_ssize_t ob_refcnt;
    PyTypeObject *ob_type;
} PyObject;
```

Тут `_PyObject_HEAD_EXTRA` - макрос, который добавляет указатели на предыдущий и следующий объект в памяти. Это позволяет сборщику мусора ходить по двухсвязному списку всех активных объектов и удалять их в случае необходимости

Для объектов переменной длины так же имеются поле `ob_size`. Такие объекты называются `PyVarObject`.

```c
typedef struct {
    PyObject ob_base;
    Py_ssize_t ob_size;
} PyVarObject;
```

Класс в python

Класс создается с помощью ключевого слова `class`:

```python
class Test:
    pass
```

Хотя он и кажется пустым, тем не менее он имеет целую охапку полей и методов, которые python создает автоматически. Их можно увидеть с помощью функции `dir()` и `var()`:

```python
dir(Test)
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__', '__forma
```

```python
vars(Test)
```

```
mappingproxy({'__module__': '__main__', '__firstlineno__': 1, '__static_attributes__': (), '__dict__
```

Чтобы создать объект класса достаточно вызвать его с синтаксисом похожим на функцию

```python
a = Test()
a
```

```
<__main__.Test object at 0x10449a510>
```

На самом деле это и является неявно вызываемыми функциями, которая вызывается каждый раз при создании объекта. А конкретно метод __new__ класса Test (выделяет память и создает объект данного класса). А так же метод __init__() объекта, который дает логику внутрь объекта (к примеру заполняет его поля). Так что семантически такой вариант будет аналогичным:

```python
a = Test.__new__(Test)

if isinstance(a, Test):
    a.__init__()

a
```

```
<__main__.Test object at 0x1044e4b90>
```

Так что если при написании класса мы определим метод __init__, тогда объект класса будет иметь соответсвующую логику::

```python
class Test:
    def __init__(self):
        print("Hello, world!")
        self.a = 123
```

```python
a = Test()
```

```
Hello, world!
```

```python
a.a
```

```
123
```

Аналогично:

```python
b = Test.__new__(Test)

if isinstance(a, Test):
    b.__init__()
```

```
Hello, world!
```

```python

b.a
```

```
123
```

Так же мы можем в рантайме изменять, добавлять и удалять атрибуты и методы

```python
b.b = "Новый аттрибут"
b.b
```

```
'Новый аттрибут'
```

```python
def func(num):
```

```
    print(f"Новый метод №{num}")

b.c = func
b.c(1)
```

```
Новый метод №1
```

При этом вызвать новый метод будет именно методом конкретного объекта:

```
a.b
a.c(2)
```

```
Traceback (most recent call last):
  File "<chunk>", line 1, in <module>
AttributeError: 'Test' object has no attribute 'b'
```

Такой подход позволяет реализовывать активную интроспекцию и рефлексию, меняя, добавляя и удаляя поля в рантайме.

Инкапсуляция

Сокрытие

При проектировании языка python исходили из концепции: “все мы взрослые люди”.
Поэтому в python нет классических `public`, `protect` и `private` полей.

Однако вместо этого есть соглашение о наименовании доступов. И PEP 8 они описываются так:

- `_single_leading_underscore`: слабый индикатор “внутреннего использования”. Например, `from M import *` не импортирует объекты, имена которых начинаются с подчеркивания.
- `single_trailing_underscore_`: используется по соглашению во избежание конфликтов с ключевым словом Python, например:
- `tkinter.Toplevel(master, class_='ClassName')`
- `__double_leading_underscore`: при именовании атрибута класса вызывает искажение имени (внутри класса FooBar `__boo` становится `_FooBar__boo`; см. ниже).
- `__double_leading_and_trailing_underscore__`: «магические» объекты или атрибуты, которые находятся в управляемых пользователем пространствах имен. Например, `__init__`, `__import__` или `__file__`. Никогда не придумывайте такие имена; используйте их только в соответствии с документацией.

Полиморфизм

Перегрузка

В python в отличии от C++ нет классической перегрузки.
В том числе потому что часть сути

```
def func(a, b):
    return a+b

func(1,2)
```

```
3
```

```python
func("1", "2")
```

```
'12'
```

Утиная типизация

Основная идея утиной типизации - это принцип “если что-то выглядит как утка, крякает утка, то это утка” и python переполнен примерами применения

Метаклассы

По сути метаклассы - это классы объектами которами являются другие классы.
Раньше мы применяли `type` как функцию, которая принимает объект и возвращает класс объекта.
Но на самом деле это класс, который при передаче одного аргумента возвращает класс, к которому относится объект

```python
class MySuperClass: ...
class MyMixin: ...

class MyClass(MySuperClass, MyMixin):
    x = 42
    def x2(self):
        return self.x * 2

MyClass()
```

```
<__main__.MyClass object at 0x10449a510>
```

С помощью конструктора `type` мы можем создать `MyClass` во время выполнения:

```python
MyClass = type('MyClass',
        (MySuperClass, MyMixin),
        {'x': 42, 'x2': lambda self: self.x * 2},
    )

MyClass()
```

```
<__main__.MyClass object at 0x10449a510>
```